

■ Electrical kW Demand Limiter (Niagara Program Object)

This guide provides a **drop■in Niagara Program Object** you can wire into a BAS graphic (Workbench) to support **electrical demand limiting** strategies.

The philosophy is the same as **VOLTTRON Intelligent Load Control (ILC)**: the automation logic **does not** live in the utility/ADR client and it **does not** live in one monolithic “master sequence.” Instead, this block:

1. **Watches a building kW meter** (or feeder meter)
2. **Computes a rolling kW average** over a demand window (15 or 30 minutes)
3. **Compares** that average to a configurable **demand limit**
4. **Ranks controllable loads** using a lightweight, explainable **hierarchical decision model** (AHP / “HRM■style” clusters)
5. Emits **simple outputs** (curtail requests + per■load priorities) that a BAS tech can wire into their own HVAC / lighting / process sequences.

■ ■ ■ Intent: This is **not** a production■ready utility compliance implementation. It's a clean ■ starting point for building teams that want a **clear contract** between “grid signal / meter” ■ and “local control algorithms.”

■ What this Program Object does (and what it does NOT do)

Does

- Provide a single place to configure **demand window**, **limit**, **deadband**, **minimum on/off**, **stagger release**.
- Provide a **ranked list** of candidates (RTUs, VAV AHUs, CHW plant stage, lighting zones, EV chargers, etc.).
- Provide BAS■friendly **slots**: numeric kW targets, booleans, and “next device to shed” strings.

Does NOT

- Directly write setpoints to field controllers (unless you explicitly map output slots into your own write blocks).
- Attempt to solve comfort constraints globally (you implement constraints locally).
- Replace your existing sequences; it just feeds them a clean signal.

■ ■ Typical California program patterns this supports

The same “demand■limiting skeleton” can be used for common CA■style demand response / pricing constructs:

1) Fast DR dispatch (event■driven)

- Event comes in: **start/stop**, **requested reduction kW**, **ramp time**
- You set:
 - ``active = true``
 - ``demandLimitKw = baselineKw - requestedReductionKw`` (or direct ``limit``)
 - optional ``eventEndTime``

2) Building kW cap window (scheduled)

- Utility says: “from 4–9pm keep the building under **X kW**”
- You set:
 - ``active = true``
 - ``demandLimitKw = X``

3) Day■ahead hourly price (shape load)

- You can re■use the same block to generate a **price■driven target kW**

- You set:
- `active = true`
- `priceSignal = \$/kWh`
- `priceToKwCurve` (simple piecewise rule)
- block converts price → kW target, then runs the same ranking + curtail loop

■ “HRM■style” decision model (AHP clusters)

VOLTTRON ILC uses **Analytic Hierarchy Process (AHP)**: weights criteria, then ranks alternatives, and repeats every few minutes.

This Program Object mirrors that idea but keeps it simple for Niagara:

- **Clusters** group similar assets (e.g., “RTUs”, “Lighting”, “EV chargers”)
- Each cluster has:
 - `clusterPriority` (importance vs other clusters)
 - `criteriaWeights` derived from a pairwise matrix (AHP)
- Each asset provides current **criteria values** (0..1 normalized):
 - `comfortPenalty`
 - `shedCapabilityKw`
 - `runtimePenalty`
 - `criticalityPenalty`
 - `recentCurtailCountPenalty`

The output is a sorted list of loads with a numeric **shedScore**.

■ Slot map (what shows up in Workbench)

Inputs

Slot	Type	Example	Notes
---	---	---	---
`enable`	Bool	true	Master enable
`active`	Bool	false	Set true during an ADR window
`demandLimitKw`	Numeric	150.0	Target building kW cap
`demandDeadbandKw`	Numeric	5.0	Avoid chatter
`demandWindowMinutes`	Numeric	15.0	Rolling average window
`controlStepSeconds`	Numeric	30.0	ProgramObject execute interval
`curtailConfirmMinutes`	Numeric	5.0	How long we must be above limit before shedding more
`curtailBreakMinutes`	Numeric	10.0	Minimum time between shed actions
`staggerRelease`	Bool	true	Release loads in waves
`releaseStepMinutes`	Numeric	2.0	Time between releasing one load and the next
`meterKw`	Numeric (link)	173.2	Link to building meter point
`externalHold`	Bool	false	Freeze state machine (operator control)
`manualOverrideMode`	Enum	AUTO	AUTO / HOLD / RELEASE_ALL

Outputs

Slot	Type	Example	Notes
---	---	---	---
`avgKw`	Numeric	168.4	Rolling average kW
`overByKw`	Numeric	18.4	`avgKw` - demandLimitKw
`state`	Enum	CURTAIN	INACTIVE / CURTAIL / HOLDING / RELEASING
`nextShedAsset`	String	RTU_3	Highest priority to shed next
`nextReleaseAsset`	String	LIGHTING_ZONE_2	Next asset to release
`shedRequest`	Bool	true	A simple “curtail now” flag for downstream sequences
`shedTargetKw`	Numeric	150.0	For downstream logic
`statusText`	String	...	Human readable

Per■asset child slots (one folder per controllable load)

Each asset appears as a child folder:

- `estimatedKw` (how much it can shed)
- `enabled`
- `comfortPenalty` (0..1)
- `criticality` (0..1)
- `runtimePenalty` (0..1)
- `recentCurtailCount`
- `shedScore` (computed)
- `shedCommand` (Bool output)
- `releaseCommand` (Bool output)

■ State machine (mirrors ILC patterns)

The logic matches the ILC “curtail / holding / releasing” pattern:

- ****INACTIVE****: not active or no target
- ****CURTAIL****: above target → compute ranking → request shed actions
- ****HOLDING****: wait for the demand window + confirm time
- ****RELEASING****: below target → release in waves (optional stagger)

■ Default tuning (good starting points)

These mirror typical ILC style knobs:

- 15 minute rolling demand window
- 5 minute confirm time (don't shed instantly)
- 10 minute break between additional shed actions
- 2 minute stagger release interval

■ Example wiring pattern (how BAS tech uses it)

1. Link `meterKw` to your ****main kW meter****.
2. During an event window:
 - set `active = true`
 - set `demandLimitKw` (from ADR client OR operator)
3. Wire the ****child asset outputs**** (`shedCommand`, `releaseCommand`) into:
 - RTU “max stages” limit
 - SAT reset bias
 - CHW plant stage inhibit
 - lighting dim level
4. Keep comfort logic local.

■ Completed Niagara Program Object (Java)

Below is a single file Program Object implementation you can paste into a Niagara module project and evolve.

<details>

<summary>■ Java: <code>KwDemandLimiterProgram.java</code> (Program Object + HRM/AHP ranking)</summary>

```
/*
 * KwDemandLimiterProgram.java
 *
 * Niagara Program Object example for electrical kW demand limiting.
 * Inspired by VOLTTRON ILC patterns: rolling average demand window,
```

```

curtail/hold/release
 * state machine, and AHP-style multi-criteria ranking (cluster + criteria
weights).
 *
 * NOTE: This is a learning / starter example. Validate thoroughly before
production use.
 */

```

```

package com.yourcompany.dsm;

```

```

import javax.baja.sys.*;
import javax.baja.control.*;
import javax.baja.status.*;
import javax.baja.nre.annotations.*;
import javax.baja.time.*;
import java.util.*;

```

```

/**
 * Drop-in Program Object:
 * - Reads a building kW meter
 * - Computes rolling avg kW
 * - Compares to demandLimitKw
 * - If above, ranks loads and issues shed/release commands
 */

```

```

@NiagaraType
@NiagaraProperty(
    name = "enable", type = "baja:Boolean", defaultValue = "true"
)
@NiagaraProperty(
    name = "active", type = "baja:Boolean", defaultValue = "false"
)
@NiagaraProperty(
    name = "demandLimitKw", type = "baja:Double", defaultValue = "150.0"
)
@NiagaraProperty(
    name = "demandDeadbandKw", type = "baja:Double", defaultValue = "5.0"
)
@NiagaraProperty(
    name = "demandWindowMinutes", type = "baja:Double", defaultValue = "15.0"
)
@NiagaraProperty(
    name = "controlStepSeconds", type = "baja:Double", defaultValue = "30.0"
)
@NiagaraProperty(
    name = "curtailConfirmMinutes", type = "baja:Double", defaultValue = "5.0"
)
@NiagaraProperty(
    name = "curtailBreakMinutes", type = "baja:Double", defaultValue = "10.0"
)
@NiagaraProperty(
    name = "staggerRelease", type = "baja:Boolean", defaultValue = "true"
)
@NiagaraProperty(
    name = "releaseStepMinutes", type = "baja:Double", defaultValue = "2.0"
)
@NiagaraProperty(
    name = "externalHold", type = "baja:Boolean", defaultValue = "false"
)
@NiagaraProperty(
    name = "manualOverrideMode",
    type = "com.yourcompany.dsm:KwDlmOverrideMode",
    defaultValue = "com.yourcompany.dsm:KwDlmOverrideMode.AUTO"
)
@NiagaraProperty(
    name = "meterKw", type = "baja:Double", defaultValue = "0.0"
)

```

```

)
@NiagaraProperty(
    name = "avgKw", type = "baja:Double", defaultValue = "0.0", flags =
Flags.READONLY
)
@NiagaraProperty(
    name = "overByKw", type = "baja:Double", defaultValue = "0.0", flags =
Flags.READONLY
)
@NiagaraProperty(
    name = "state", type = "com.yourcompany.dsm:KwDlmState",
    defaultValue = "com.yourcompany.dsm:KwDlmState.INACTIVE", flags = Flags.READONLY
)
@NiagaraProperty(
    name = "nextShedAsset", type = "baja:String", defaultValue = "", flags =
Flags.READONLY
)
@NiagaraProperty(
    name = "nextReleaseAsset", type = "baja:String", defaultValue = "", flags =
Flags.READONLY
)
@NiagaraProperty(
    name = "shedRequest", type = "baja:Boolean", defaultValue = "false", flags =
Flags.READONLY
)
@NiagaraProperty(
    name = "shedTargetKw", type = "baja:Double", defaultValue = "0.0", flags =
Flags.READONLY
)
@NiagaraProperty(
    name = "statusText", type = "baja:String", defaultValue = "", flags =
Flags.READONLY
)
public class KwDemandLimiterProgram extends BProgramObject
{
    /*+ ----- BEGIN BAJA AUTO GENERATED CODE ----- +*/
    /*@ $com.yourcompany.dsm.KwDemandLimiterProgram(1234567890)1.0$ @*/
    public static final Property enable = newProperty(0, true, null);
    public boolean getEnable() { return getBoolean(enable); }
    public void setEnable(boolean v) { setBoolean(enable, v, null); }

    public static final Property active = newProperty(0, false, null);
    public boolean getActive() { return getBoolean(active); }
    public void setActive(boolean v) { setBoolean(active, v, null); }

    public static final Property demandLimitKw = newProperty(0, 150.0, null);
    public double getDemandLimitKw() { return getDouble(demandLimitKw); }
    public void setDemandLimitKw(double v) { setDouble(demandLimitKw, v, null); }

    public static final Property demandDeadbandKw = newProperty(0, 5.0, null);
    public double getDemandDeadbandKw() { return getDouble(demandDeadbandKw); }
    public void setDemandDeadbandKw(double v) { setDouble(demandDeadbandKw, v,
null); }

    public static final Property demandWindowMinutes = newProperty(0, 15.0, null);
    public double getDemandWindowMinutes() { return getDouble(demandWindowMinutes); }
}
    public void setDemandWindowMinutes(double v) { setDouble(demandWindowMinutes, v,
null); }

    public static final Property controlStepSeconds = newProperty(0, 30.0, null);
    public double getControlStepSeconds() { return getDouble(controlStepSeconds); }
    public void setControlStepSeconds(double v) { setDouble(controlStepSeconds, v,
null); }
}

```

```

    public static final Property curtailConfirmMinutes = newProperty(0, 5.0, null);
    public double getCurtailConfirmMinutes() { return
getDouble(curtailConfirmMinutes); }
    public void setCurtailConfirmMinutes(double v) {
setDouble(curtailConfirmMinutes, v, null); }

    public static final Property curtailBreakMinutes = newProperty(0, 10.0, null);
    public double getCurtailBreakMinutes() { return getDouble(curtailBreakMinutes);
}
    public void setCurtailBreakMinutes(double v) { setDouble(curtailBreakMinutes, v,
null); }

    public static final Property staggerRelease = newProperty(0, true, null);
    public boolean getStaggerRelease() { return getBoolean(staggerRelease); }
    public void setStaggerRelease(boolean v) { setBoolean(staggerRelease, v, null);
}

    public static final Property releaseStepMinutes = newProperty(0, 2.0, null);
    public double getReleaseStepMinutes() { return getDouble(releaseStepMinutes); }
    public void setReleaseStepMinutes(double v) { setDouble(releaseStepMinutes, v,
null); }

    public static final Property externalHold = newProperty(0, false, null);
    public boolean getExternalHold() { return getBoolean(externalHold); }
    public void setExternalHold(boolean v) { setBoolean(externalHold, v, null); }

    public static final Property manualOverrideMode = newProperty(0,
KwDlmOverrideMode.AUTO, null);
    public KwDlmOverrideMode getManualOverrideMode() { return
(KwDlmOverrideMode)get(manualOverrideMode); }
    public void setManualOverrideMode(KwDlmOverrideMode v) { set(manualOverrideMode,
v, null); }

    public static final Property meterKw = newProperty(0, 0.0, null);
    public double getMeterKw() { return getDouble(meterKw); }
    public void setMeterKw(double v) { setDouble(meterKw, v, null); }

    public static final Property avgKw = newProperty(Flags.READONLY, 0.0, null);
    public double getAvgKw() { return getDouble(avgKw); }
    private void setAvgKw(double v) { setDouble(avgKw, v, null); }

    public static final Property overByKw = newProperty(Flags.READONLY, 0.0, null);
    public double getOverByKw() { return getDouble(overByKw); }
    private void setOverByKw(double v) { setDouble(overByKw, v, null); }

    public static final Property state = newProperty(Flags.READONLY,
KwDlmState.INACTIVE, null);
    public KwDlmState getState() { return (KwDlmState)get(state); }
    private void setState(KwDlmState v) { set(state, v, null); }

    public static final Property nextShedAsset = newProperty(Flags.READONLY, "",
null);
    public String getNextShedAsset() { return getString(nextShedAsset); }
    private void setNextShedAsset(String v) { setString(nextShedAsset, v, null); }

    public static final Property nextReleaseAsset = newProperty(Flags.READONLY, "",
null);
    public String getNextReleaseAsset() { return getString(nextReleaseAsset); }
    private void setNextReleaseAsset(String v) { setString(nextReleaseAsset, v,
null); }

    public static final Property shedRequest = newProperty(Flags.READONLY, false,
null);
    public boolean getShedRequest() { return getBoolean(shedRequest); }
    private void setShedRequest(boolean v) { setBoolean(shedRequest, v, null); }

```

```

    public static final Property shedTargetKw = newProperty(Flags.READONLY, 0.0,
null);
    public double getShedTargetKw() { return getDouble(shedTargetKw); }
    private void setShedTargetKw(double v) { setDouble(shedTargetKw, v, null); }

    public static final Property statusText = newProperty(Flags.READONLY, "", null);
    public String getStatusText() { return getString(statusText); }
    private void setStatusText(String v) { setString(statusText, v, null); }

    /*+ ----- END BAJA AUTO GENERATED CODE ----- +*/

    // -----
    // Internal state
    // -----
    private final Deque<Double> kwSamples = new ArrayDeque<>();
    private long lastSampleMs = 0L;

    private long lastCurtailMs = 0L;
    private long lastReleaseMs = 0L;
    private long aboveLimitSinceMs = 0L;

    // "HRM" = cluster -> assets
    private final List<Cluster> clusters = new ArrayList<>();

    @Override
    public void started()
    {
        super.started();

        // You can replace these with child folder discovery; this is a starter "in-
code" sample.
        clusters.clear();
        clusters.add(Cluster.sampleRtuCluster());
        clusters.add(Cluster.sampleLightingCluster());

        // Execute at the user-configured step.
        BRelTime period = BRelTime.makeSeconds((long)Math.max(getControlStepSeconds(),
5.0));
        setExecutePeriod(period);
    }

    @Override
    public void doExecute() throws Exception
    {
        if (!getEnable())
        {
            resetOutputs("Disabled");
            return;
        }

        // Manual override mode: HOLD or RELEASE_ALL
        if (getManualOverrideMode() == KwdlmOverrideMode.RELEASE_ALL)
        {
            requestReleaseAll("Manual RELEASE_ALL");
            return;
        }
        if (getManualOverrideMode() == KwdlmOverrideMode.HOLD || getExternalHold())
        {
            setState(KwdlmState.HOLDING);
            setStatusText("HOLD (manual/external)");
            setShedRequest(false);
            return;
        }
    }

```

```

if (!getActive() || getDemandLimitKw() <= 0.0)
{
    resetOutputs("Inactive or no demand target");
    return;
}

// 1) Update rolling average
updateRollingAverage(getMeterKw());

double avg = getAvgKw();
double over = avg - getDemandLimitKw();
setOverByKw(over);
setShedTargetKw(getDemandLimitKw());

// 2) Decide whether we're above or below
boolean above = over > getDemandDeadbandKw();
boolean below = over < -getDemandDeadbandKw();

long now = System.currentTimeMillis();

if (above)
{
    if (aboveLimitSinceMs == 0L) aboveLimitSinceMs = now;

    // Confirm above limit long enough before shedding more
    long confirmMs = (long)(getCurtailConfirmMinutes() * 60_000.0);
    if (now - aboveLimitSinceMs < confirmMs)
    {
        setState(KwDlmState.HOLDING);
        setStatusText("Above limit but waiting confirm: " + minutesLeft(confirmMs
- (now - aboveLimitSinceMs)) + " min");
        setShedRequest(false);
        return;
    }

    // Enforce break between curtail actions
    long breakMs = (long)(getCurtailBreakMinutes() * 60_000.0);
    if (lastCurtailMs != 0L && (now - lastCurtailMs) < breakMs)
    {
        setState(KwDlmState.HOLDING);
        setStatusText("Above limit; curtail break active: " + minutesLeft(breakMs
- (now - lastCurtailMs)) + " min");
        setShedRequest(true); // keep downstream logic aware we are in curtail
mode
        return;
    }

    // 3) Rank assets (AHP / HRM-style)
    List<RankedAsset> ranked = rankAssets();

    // 4) Choose next shed target
    RankedAsset next = ranked.isEmpty() ? null : ranked.get(0);
    if (next == null)
    {
        setState(KwDlmState.CURTAIL);
        setStatusText("Above limit but no enabled assets available to shed.");
        setNextShedAsset("");
        setShedRequest(true);
        return;
    }

    setState(KwDlmState.CURTAIL);
    setNextShedAsset(next.assetId);
    setStatusText("Curtail: avg=" + fmt(avg) + "kW limit=" +
fmt(getDemandLimitKw()) + "kW; next=" + next.assetId + " score=" +

```

```

fmt(next.score));

    // 5) Emit "request" signals (BAS wires these into their own sequences)
    setShedRequest(true);
    issueShedCommand(next);

    lastCurtailMs = now;
    lastReleaseMs = 0L;
    return;
}

// If below, attempt releasing (optional stagger)
if (below)
{
    aboveLimitSinceMs = 0L;
    setShedRequest(false);

    if (!getStaggerRelease())
    {
        requestReleaseAll("Below limit (release all)");
        return;
    }

    long stepMs = (long)(getReleaseStepMinutes() * 60_000.0);
    if (lastReleaseMs != 0L && (now - lastReleaseMs) < stepMs)
    {
        setState(KwDlmState.RELEASING);
        setStatusText("Releasing (stagger): waiting " + minutesLeft(stepMs - (now
- lastReleaseMs)) + " min");
        return;
    }

    RankedAsset toRelease = pickNextRelease();
    if (toRelease == null)
    {
        setState(KwDlmState.RELEASING);
        setStatusText("Below limit; nothing left to release.");
        setNextReleaseAsset("");
        return;
    }

    setState(KwDlmState.RELEASING);
    setNextReleaseAsset(toRelease.assetId);
    setStatusText("Release: next=" + toRelease.assetId);

    issueReleaseCommand(toRelease);
    lastReleaseMs = now;
    return;
}

// Within deadband: hold state
aboveLimitSinceMs = 0L;
setState(KwDlmState.HOLDING);
setShedRequest(false);
setStatusText("Within deadband: avg=" + fmt(avg) + "kW limit=" +
fmt(getDemandLimitKw()) + "kW");
}

// -----
// Rolling average (simple deque)
// -----
private void updateRollingAverage(double currentKw)
{
    long now = System.currentTimeMillis();
    long stepMs = (long)(Math.max(getControlStepSeconds(), 5.0) * 1000.0);

```

```

// Sample at execute period; ignore if executed too fast
if (lastSampleMs != 0L && (now - lastSampleMs) < (stepMs * 0.8)) return;
lastSampleMs = now;

kwSamples.addLast(currentKw);

int maxSamples = (int)Math.max(1.0, (getDemandWindowMinutes() * 60.0) /
Math.max(getControlStepSeconds(), 5.0));
while (kwSamples.size() > maxSamples) kwSamples.removeFirst();

double sum = 0.0;
for (double v : kwSamples) sum += v;
setAvgKw(sum / kwSamples.size());
}

// -----
// Ranking (AHP / "HRM style")
// -----
private List<RankedAsset> rankAssets()
{
    List<RankedAsset> out = new ArrayList<>();

    for (Cluster c : clusters)
    {
        if (!c.enabled) continue;

        // weights already derived from pairwise matrix (AHP), cached in cluster
        double[] w = c.criteriaWeights;

        for (Asset a : c.assets)
        {
            if (!a.enabled) continue;

            // "input matrix" is simplified: we assume each criterion is already
normalized 0..1
            double score =
                a.comfortPenalty      * w[0] +
                a.shedCapability       * w[1] +
                a.runtimePenalty       * w[2] +
                a.criticalityPenalty   * w[3] +
                a.recentCurtailPenalty * w[4];

            score = score * c.clusterPriority;

            out.add(new RankedAsset(a.assetId, score, c.clusterId));
        }
    }

    out.sort((x, y) -> Double.compare(y.score, x.score));
    return out;
}

private void issueShedCommand(RankedAsset a)
{
    Asset asset = findAsset(a.assetId);
    if (asset == null) return;

    asset.shedCommand = true;
    asset.releaseCommand = false;
    asset.recentCurtailCount += 1;
}

private void issueReleaseCommand(RankedAsset a)
{

```

```

    Asset asset = findAsset(a.assetId);
    if (asset == null) return;

    asset.shedCommand = false;
    asset.releaseCommand = true;
}

private RankedAsset pickNextRelease()
{
    // Release in reverse order: lowest score first (or those currently shed)
    List<RankedAsset> ranked = rankAssets();
    Collections.reverse(ranked);

    for (RankedAsset r : ranked)
    {
        Asset a = findAsset(r.assetId);
        if (a != null && a.wasShedRecently())
        {
            return r;
        }
    }
    return null;
}

private void requestReleaseAll(String reason)
{
    for (Cluster c : clusters)
        for (Asset a : c.assets)
        {
            a.shedCommand = false;
            a.releaseCommand = true;
        }

    setState(KwDlmState.RELEASING);
    setShedRequest(false);
    setNextShedAsset("");
    setNextReleaseAsset("");
    setStatusText(reason + " | avg=" + fmt(getAvgKw()) + "kw");
}

private void resetOutputs(String reason)
{
    aboveLimitSinceMs = 0L;
    lastCurtailMs = 0L;
    lastReleaseMs = 0L;

    setState(KwDlmState.INACTIVE);
    setShedRequest(false);
    setNextShedAsset("");
    setNextReleaseAsset("");
    setOverByKw(0.0);
    setShedTargetKw(0.0);
    setStatusText(reason);

    // also clear commands
    for (Cluster c : clusters)
        for (Asset a : c.assets)
        {
            a.shedCommand = false;
            a.releaseCommand = false;
        }
}

private Asset findAsset(String assetId)
{

```

```

    for (Cluster c : clusters)
        for (Asset a : c.assets)
            if (a.assetId.equals(assetId)) return a;
    return null;
}

private static String fmt(double v)
{
    return String.format(Locale.US, "%.2f", v);
}

private static String minutesLeft(long ms)
{
    return String.format(Locale.US, "%.1f", ms / 60000.0);
}

private static final class Cluster
{
    final String clusterId;
    boolean enabled = true;
    double clusterPriority;           // 0..1
    double[] criteriaWeights;        // 5 criteria
    final List<Asset> assets = new ArrayList<>();

    Cluster(String id, double clusterPriority, double[] weights)
    {
        this.clusterId = id;
        this.clusterPriority = clusterPriority;
        this.criteriaWeights = normalize(weights);
    }

    static Cluster sampleRtuCluster()
    {
        double[] w = new double[] {0.35, 0.25, 0.15, 0.20, 0.05};
        Cluster c = new Cluster("RTU", 0.8, w);
        c.assets.add(new Asset("RTU_1", 1.0));
        c.assets.add(new Asset("RTU_2", 1.0));
        c.assets.add(new Asset("RTU_3", 1.0));
        return c;
    }

    static Cluster sampleLightingCluster()
    {
        double[] w = new double[] {0.15, 0.45, 0.10, 0.25, 0.05};
        Cluster c = new Cluster("LIGHTING", 0.2, w);
        c.assets.add(new Asset("LIGHTING_ZONE_1", 0.3));
        c.assets.add(new Asset("LIGHTING_ZONE_2", 0.3));
        return c;
    }

    private static double[] normalize(double[] w)
    {
        double s = 0.0;
        for (double v : w) s += v;
        double[] out = new double[w.length];
        for (int i = 0; i < w.length; i++) out[i] = (s == 0.0) ? 0.0 : (w[i] / s);
        return out;
    }
}

private static final class Asset
{
    final String assetId;
    boolean enabled = true;

```

```

// Normalized inputs (0..1)
double comfortPenalty = 0.2;
double shedCapability = 0.8;
double runtimePenalty = 0.3;
double criticalityPenalty = 0.3;
double recentCurtailPenalty = 0.1;

// Outputs
boolean shedCommand = false;
boolean releaseCommand = false;

// State
int recentCurtailCount = 0;

// Informational estimate
double estimatedKw;

Asset(String id, double estimatedKw)
{
    this.assetId = id;
    this.estimatedKw = estimatedKw;
}

boolean wasShedRecently()
{
    return recentCurtailCount > 0;
}
}

private static final class RankedAsset
{
    final String assetId;
    final double score;
    final String clusterId;

    RankedAsset(String assetId, double score, String clusterId)
    {
        this.assetId = assetId;
        this.score = score;
        this.clusterId = clusterId;
    }
}
}

```

</details>

■ Suggested project layout (Niagara module)

```

demand_side_management/
  kw_demand_limiter/
    README_KW_DEMAND_LIMITER.md
  module/
    src/
      com/yourcompany/dsm/
        KwdemandLimiterProgram.java
        KwdlmState.java
        KwdlmOverrideMode.java

```

■ Next upgrades (if you want to make it “real”)

- Discover assets dynamically as child folders rather than hardcoding.
- Compute criteria live:
- $\text{comfort} = |\text{zoneTemp} - \text{setpoint}|$
- $\text{runtime} = \text{minutes since last stage change}$
- $\text{criticality} = \text{room type or owner weighting}$
- $\text{shedCapability} = \text{rated kW or measured delta after sheds}$
- Replace example weights with true AHP pairwise matrices + consistency checks.
- Add “global override” / “local override” points (operator safety) like ILC does.
- Add a *billing cycle peak forecast* module if you want true demand ■ charge optimization.

Last updated: 2026-01-14